◇ Lightning Labs

pi-402

Paying for Kimi K3 with sats

The strongest open-weights model on earth, metered by the token and settled over Lightning. No account, no API key, no credit card.

Olaoluwa Osuntokun — Lightning Labs

The Best Open Model Is Behind a Credit Card

Moonshot released the Kimi K3 weights on **2026-07-27**, the #1 open model on Artificial Analysis. Anyone may download it, and almost nobody can run it: it wants a rack of GB300s. So "open weights" means "open to whoever will sell you inference," and that gate isn't the weights, it's the payment rail.

2.8T	16/896	1M	vision	1.5 TB
parameters	experts active	token context	natively multimodal	the download

What the card rail costs you

An account and a fraud check, upstream of the first token. Declines by geography and by bank. No card for an autonomous agent, and handing it yours makes it you. Card economics can't clear a $0.01 request.

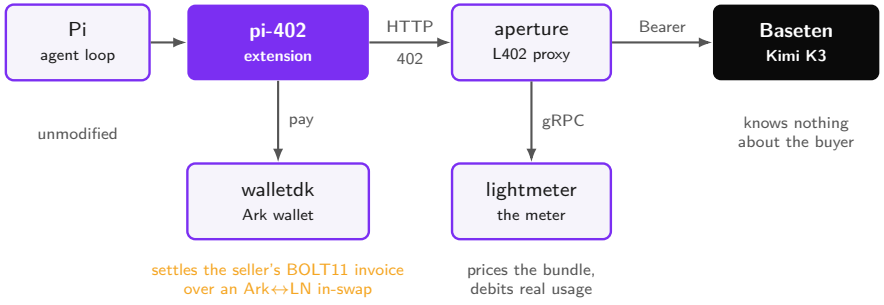
What sats give you instead

The payment *is* the credential: no identity to establish, leak, or revoke. The buyer's country is not an input. An agent holds a wallet, and the balance you fund is the blast radius. Sub-cent amounts clear.

The scarce thing was never the model, it's **permission to pay for it**.

What We Built

A coding agent that pays per call. The buyer is a Pi extension, the seller is an aperture proxy holding the model credential, and the two meet over L402.



Two properties fall out of this shape. **Pi core is never modified**, because every hook is a documented extension point. And **no API key ever reaches the buyer**, because aperture injects the model credential server-side. The buyer holds sats; the seller holds the key.

Why Pi: Payment Lives Under the Model Call

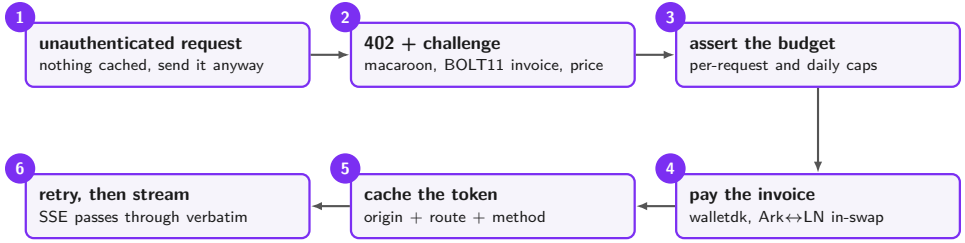
Payment has to sit on the *transport*, beneath the model call. That rules out most agents: if one owns its HTTP client and won't let you replace it, there's nowhere to put a payment loop short of forking it. Pi's `registerProvider` takes a `streamSimple` that builds its own client, so we hand it a `fetch` that knows how to pay.

What a payment layer needs	Pi hook	How we use it
A paying transport	`registerProvider`	`streamSimple` builds an OpenAI client whose `fetch` is `payingFetch`
Point at the proxy, declare the model	provider config	`baseUrl` is the L402 proxy; `models:[...]` declares `kimi-k3`
Hold a wallet for the session	`on("session_start")`	load and unlock the walletdk wallet once, then reuse it
Fund and inspect, out of band	`registerCommand`	`/wallet /deposit /balance /receipts`
Surface a declined payment	provider `error` event	budget rejections reach the user as a normal stream error

Paid inference, **no change to Pi**, and **the agent core never learns money exists**.

L402: Two Primitives, One Loop

A macaroon (attenuatable, offline-verifiable authorization) and a payment preimage (atomic proof of settlement). The macaroon's identifier commits to a payment hash H; paying the invoice for H reveals r with $\mathrm{sha256}(r) = H$; the client presents `macaroon:r`. So **a valid credential cannot exist without settlement**, and settlement reveals exactly the secret that validates it.

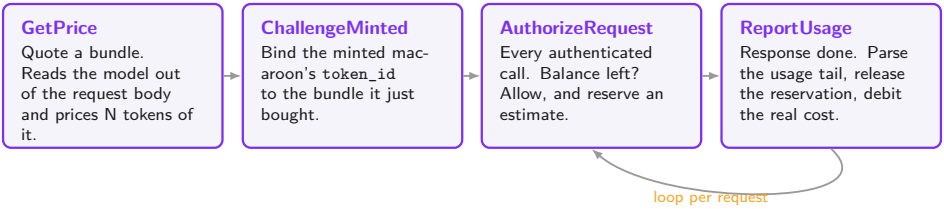


Bootstrapping is free Step 1 costs nothing: the challenge and invoice are handed out unauthenticated, and `/deposit` never touches a model. A funded wallet is the only precondition.

An Ark wallet, not a node No channels, no inbound liquidity, nothing kept online. Fund a boarding address; an in-swap turns a VTXO into a Lightning payment.

Metering Aperture: Four Call-Outs, Shipping Today

Aperture mints challenges, verifies credentials, and forwards. What it can't do is charge by the token, since it has no idea what an LLM is. So we didn't teach it: a separate daemon implements the `pricesrpc.Prices` gRPC service and owns all cost analysis.

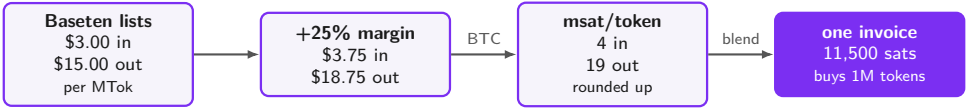


The move that makes it work `AuthorizeRequest` returning *deny* is how "you've used your million tokens" gets expressed. Aperture mints a **fresh 402**, the buyer loop pays it exactly as it paid the first, and the session continues. Bundle exhaustion and cold start are the same code path.

Two daemons `meterd` is the reference pricer in the aperture tree. `lightmeter` is the production controller: it embeds `meterd` and adds model discovery, USD to msat pricing, and a sqlite usage ledger.

The Macaroon Is the Meter

We kept the macaroon, and that's the decision worth defending: the obvious alternative, using the preimage directly as a bearer token, gives up the thing we want. L402's `token_id` is already derived from the macaroon, so the meter keys balances on it and we needed no new session concept: **the credential *is* the session**. Unlike a bare preimage, a macaroon can be **attenuated** (narrowed to a model or a budget, without the root key) and so **delegated** to a sub-agent.



A bundle is priced at the *blended* rate, the average of both directions. Debits are then **direction-weighted** against that blend, so the margin holds regardless of the buyer's prompt/completion mix, and rounding always goes *up*, since a rate that rounds to zero is a free model.

The honest trade-off. Prepaid metering relaxes atomicity from per-request to *per-bundle*: the payment is still proven by its preimage, but the requests drawing it down are on credit. Closing that gap is next.

Demo

Signet. A web wallet, an invoice, and Kimi K3.

Kimi K3 provisioned on Baseten, `/fund` paid from the Wavelength web wallet, then prompt: 402, swap, preimage, stream. Close on `/receipts`.

Every layer is real: real swap, real invoice, real preimage, real model, real usage debit.

Next: Furthering L402, Not Replacing It

Keep the atomic core, bring the session economics in. Every row below is a caveat or a rail, not a redesign.

Phase	What	Why	Payable now?
0	Session caveats + bound descriptor	Bundle, model, and budget become caveats over a committed JSON descriptor	yes
1	Signed usage receipts	The server signs each tick, so the drawdown stops being trusted	yes
1	Proof-of-possession caveat	A session pubkey, so a long-lived credential isn't a pure bearer secret	yes
2	VHTLC deposit with a refund branch	Unspent balance reclaimable on timeout: real escrow, not an IOU	wallet work
3	BOLT12 payment plane	An offer is a stable endpoint, so top-ups leave the request path	needs BOLT12

Phase 1 matters most: a signed receipt chain makes a billing discrepancy *provable* rather than a disagreement. Nearer term: land aperture#247 and move to mainnet on live rates.

Sats are enough.

The most capable open-weights model in the world is a 1.5 TB download almost nobody can run, so access runs through whoever will sell you inference. That gate has been a credit card.

It doesn't have to be. A wallet with sats, an L402 challenge, and a preimage get you the same tokens: no account, no key, no permission.